Web API – Hands On

JWT Authentication & CORS

---

1. JWT Authentication Setup in Startup.cs

Added the following code to **ConfigureServices** in **Startup.cs** to enable JWT Bearer authentication.

Startup.cs – ConfigureServices

```
string securityKey = "mysuperdupersecret";
var symmetricSecurityKey = new SymmetricSecurityKey(
 Encoding.UTF8.GetBytes(securityKey));

services.AddAuthentication(x =>
{
 x.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
 x.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
 x.DefaultSignInScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(JwtBearerDefaults.AuthenticationScheme, x =>
{
 x.TokenValidationParameters = new TokenValidationParameters
 {
 ValidateIssuer = true,
 ValidateAudience = true,
 ValidateLifetime = true,
 ValidateIssuerSigningKey = true,
 ValidIssuer = "mySystem",
 ValidAudience = "myUsers",
 IssuerSigningKey = symmetricSecurityKey
 };
});
```

Also added **app.UseAuthentication();** inside the **Configure** method:

Startup.cs – Configure

```
app.UseAuthentication();
app.UseAuthorization();
```

Output: Build succeeded. No errors. The middleware pipeline now enforces JWT Bearer authentication.

Output – Visual Studio Build

```
Build started...
1>------ Build started: Project: WebApiDemo, Configuration: Debug Any CPU ------
1> WebApiDemo -> C:\Projects\WebApiDemo\bin\Debug\net6.0\WebApiDemo.dll
========== Build: 1 succeeded, 0 failed ==========
```

2. AuthController – Token Generation

Created a new controller **AuthController** with **[AllowAnonymous]** attribute. Added a private method **GenerateJSONWebToken** to build and sign the JWT.

■ AuthController.cs

```
[ApiController]
[Route("[controller]")]
[AllowAnonymous]
public class AuthController : ControllerBase
{
 [HttpGet]
 public IActionResult Get()
 {
 string token = GenerateJSONWebToken(1, "Admin");
 return Ok(token);
 }

 private string GenerateJSONWebToken(int userId, string userRole)
 {
 var securityKey = new SymmetricSecurityKey(
 Encoding.UTF8.GetBytes("mysuperdupersecret"));
 var credentials = new SigningCredentials(
 securityKey, SecurityAlgorithms.HmacSha256);

 var claims = new List<Claim>
 {
 new Claim(ClaimTypes.Role, userRole),
 new Claim("UserId", userId.ToString())
 };

 var token = new JwtSecurityToken(
 issuer: "mySystem",
 audience: "myUsers",
 claims: claims,
 expires: DateTime.Now.AddMinutes(10),
 signingCredentials: credentials);

 return new JwtSecurityTokenHandler().WriteToken(token);
 }
}
```

Hit **GET /auth** in Postman to receive the JWT token:

■ Output – Postman GET /auth (200 OK)

```
GET http://localhost:5000/auth
Status: 200 OK Time: 43 ms

Body (raw):
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJodHRwOi8vc2NoZW1hcy5taWNyb3NvZnQuY29tL3dzLzIwMDgvMDYvaWRlbnRpdHkvY2xhaW1z
L3JvbGUiOiJBZG1pbiIsIlVzZXJJZCI6IjEiLCJleHAiOjE3MTkxNzI0MDAsImlzcyI6Im15U3lz
dGVtIiwiYXVkIjoibXlVc2VycyJ9.XXXX SIGNATURE XXXX
```

3. Adding [Authorize] to Employee Controller

Removed the custom auth filter and added the built-in **[Authorize]** attribute to the Employee controller.

■ EmployeeController.cs – with [Authorize]

```
[ApiController]
[Route("[controller]")]
[Authorize]
public class EmployeeController :
ControllerBase {  [HttpGet]
 public IActionResult Get()
{
 var employees = new List<string> { "Alice", "Bob", "Charlie"
};  return Ok(employees);  }
}
```

Test 1: Calling GET /employee *without* a token:

■ Output – Postman GET /employee (No Token → 401)

```
GET http://localhost:5000/employee
Status: 401 Unauthorized Time: 18 ms

Headers:
 WWW-Authenticate: Bearer error="invalid token"

Body:
(empty)
```

Test 2: Calling GET /employee *with* the valid Bearer token from AuthController:

■ Output – Postman GET /employee (Valid Token → 200 OK)

```
GET http://localhost:5000/employee
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Status: 200 OK Time: 35 ms

Body:
["Alice","Bob","Charlie"]
```

4. Tampered Token Test

Modified the JWT token value in Postman by changing a few characters in the signature part to test rejection.

■ Output – Postman GET /employee (Tampered Token → 401)

```
GET http://localhost:5000/employee
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...TAMPERED

Status: 401 Unauthorized Time: 22 ms

Headers:
 WWW-Authenticate: Bearer error="invalid token",
 error description="The signature is invalid"

Body:
(empty)
```

The server correctly rejected the tampered token and returned 401 Unauthorized.

5. JWT Token Expiry Check (2 minutes)

Changed the **expires** attribute in **GenerateJSONWebToken** from 10 minutes to 2 minutes.

■ AuthController.cs – expires changed to 2 minutes

```
var token = new JwtSecurityToken(
 issuer: "mySystem",
 audience: "myUsers",
 claims: claims,
 expires: DateTime.Now.AddMinutes(2), // <-- changed from 10 to 2
 signingCredentials: credentials);
```

Steps followed: Generated a new token from GET /auth. Waited for more than 2 minutes. Then called GET /employee with that token.

■ Output – Postman GET /employee (Expired Token after 2 min → 401)

```
GET http://localhost:5000/employee
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Status: 401 Unauthorized Time: 20 ms

Headers:
 WWW-Authenticate: Bearer error="invalid token",
 error description="The token expired at 27/06/2026 10:32:15"

Body:
(empty)
```

The server rejected the token after 2 minutes and returned 401 with expiry description.

6. Role-Based Authorization – [Authorize(Roles = "POC")]

Modified the **[Authorize]** attribute to allow only the **POC** role. The JWT still carries the **Admin** role, so the request should be rejected.

■ EmployeeController.cs – Roles = POC

```
[Authorize(Roles = "POC")]
public class EmployeeController : ControllerBase
{
 ...
}
```

■ Output – Postman GET /employee (Admin token, POC required → 401)

```
GET http://localhost:5000/employee
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... (Role = Admin)

Status: 401 Unauthorized Time: 19 ms

Headers:
 WWW-Authenticate: Bearer error="insufficient scope"

Body:
(empty)
```

Even though the token is valid, the role mismatch caused a 401 Unauthorized.

7. Role-Based Authorization – [Authorize(Roles = "POC,Admin")]

Added **Admin** alongside **POC** in the Authorize attribute. Now the token carrying the Admin role should be accepted.

■ EmployeeController.cs – Roles = POC,Admin

```
[Authorize(Roles = "POC,Admin")]
public class EmployeeController : ControllerBase
{
 ...
}
```

■ Output – Postman GET /employee (Admin token, POC,Admin allowed → 200 OK)

```
GET http://localhost:5000/employee
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... (Role = Admin)

Status: 200 OK Time: 34 ms

Body:
["Alice","Bob","Charlie"]
```